CSCI 3060U - Phase 5

Back End Unit Testing - Test Report

Group Members

Mansi Kandoi - 100821953
Sneh Patel - 100950609
Jolly Soni - 100876863

Table of Contents

CSCI 3060U - Phase 5 ..... *1*
Back End Unit Testing - Test Report *1*
1. Overview *2*
2. Method 1: _apply_deposit() - Statement Coverage *2*
- **Method Description** 2
- **Statements Identified** 2
- **Test Case Table** 2

3. Method 2: process_transactions() - Decision and Loop Coverage *4*
- **Method Description** 4
- **Loop and Decisions Identified** 4
- **Test Case Table** 4

4. Test Execution Results *6*
5. Failures Found *6*
6. Summary *6*
7. GitHub Repository: *6*

1. Overview

This report covers the white-box unit testing conducted on the Phase 4 Back End. We chose two methods from TransactionProcessor in process.py:

- Method 1: _apply_deposit() - tested with Statement Coverage (5 test cases, TC1-TC5)
- Method 2: process_transactions() - tested with Decision and Loop Coverage (11 test cases, TC6-TC16)

All 16 tests were run using pytest. No failures were found.

2. Method 1: _apply_deposit() - Statement Coverage

Method Description

_apply_deposit() handles deposit transactions. It looks up the account by number, returns early if the account doesn't exist or is disabled, otherwise credits the deposit amount, increments the transaction count, then calls _apply_transaction_fee(). That helper deducts 0.05 for SP accounts and 0.10 for NP accounts, but only if the balance is enough to cover the fee.

Statements Identified

The following executable statements were identified for coverage:

- S1 - _get_existing_account() called to look up the account
- S2 - Guard: if account is None or not active (two conditions, one statement)
- S3 - Early return when the guard fires (account missing or disabled)
- S4 - account.credit(transaction.amount) - funds added
- S5 - account.increment_transaction_count()
- S6 - _apply_transaction_fee(account) called
- S7 - SP branch inside the fee helper: fee = 0.05
- S8 - NP else branch inside the fee helper: fee = 0.10 (S7 and S8 are mutually exclusive)
- S9 - if balance >= fee (true): account.debit(fee) runs
- S10 - if balance >= fee (false): fee deduction is skipped
-

Test Case Table

TC#	Test Function	Description	Statements Hit	Expected Result	Pass/Fail
TC1	test_apply_deposit_active_student_ account_credits_amount_and_applies_sp_fee	Active SP account, deposit $50	S1, S2(F), S4, S5, S6, S7(T), S9(T)	Balance = 149.95 tx count = 1	**PASS**
TC2	test_apply_deposit_active_non_student_ account_applies_np_fee	Active NP account, deposit $50	S8 (NP else branch)	Balance = 149.90 tx count = 1	**PASS**
TC3	test_apply_deposit_disabled_ account_is_rejected	Account is disabled - reject	S2(T), S3 early return	Balance = 100.00 tx count = 0	**PASS**
TC4	test_apply_deposit_unknown_account_ number_does_nothing	Account number not in system	S2(T), S3 early return	Other acct unchanged	**PASS**

TC#	Test Function	Description	Statements Hit	Expected Result	Pass/Fail
TC5	test_apply_deposit_fee_is_skipped_ when_balance_is_below_fee	Balance (0.03) < SP fee (0.05) after deposit	S9(F) - fee skipped	Balance = 0.03 tx count = 1	**PASS**

3. Method 2: process_transactions() - Decision and Loop Coverage

Method Description

process_transactions() loops through a list of Transaction objects and calls _dispatch_transaction() for each one. That dispatcher uses an if/elif/else chain to route each transaction to the right handler based on the code. Once all transactions are applied, accounts are returned in sorted order. We picked this method because it has a clear for loop and nine distinct branch conditions, which makes it a good fit for decision and loop coverage.

Loop and Decisions Identified

Loop:

- L1 - for transaction in transactions (tested at 0, 1, and 2+ iterations)

Decisions (if/elif/else chain inside _dispatch_transaction()):

- D1- code == "01" → withdrawal
- D2 - code == "02" → transfer
- D3 - code == "03" → paybill
- D4 - code == "04" → deposit
- D5 - code == "05" → create account
- D6 - code == "06" → delete account
- D7 - code == "07" → disable account
- D8 - code == "08" → change plan
- D9 - else → unknown or unrecognized code

Because this is an if/elif chain, taking the true branch of D(n) also implicitly exercises the false branch of all earlier conditions D1 through D(n−1). So 11 tests are enough to cover both sides of every decision and all three loop iteration counts.

Test Case Table

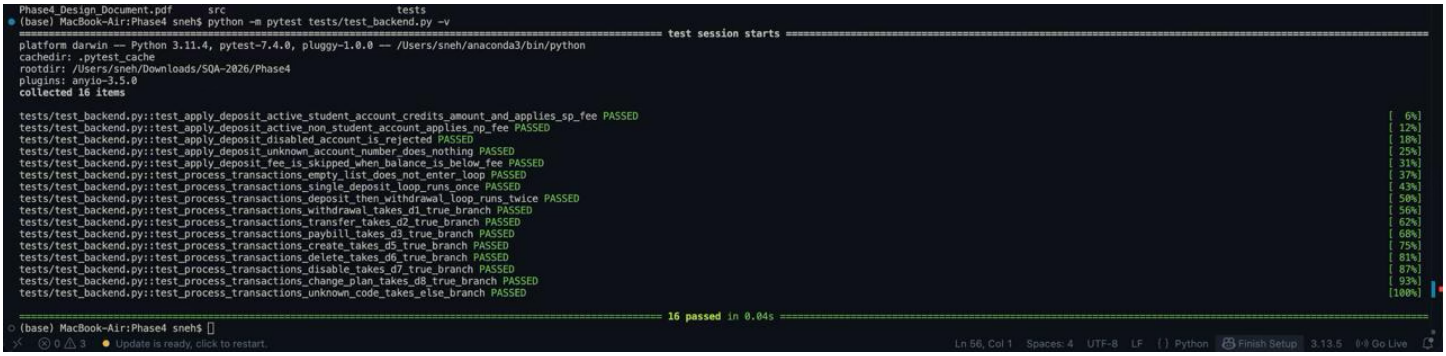
TC#	Test Function	Code	Decision / Loop	Expected Result	Pass/ Fail
TC6	test_process_transactions_empty_list_does_not_enter_loop	-	L1=0: loop body never entered	Accounts sorted, unchanged	**PASS**
TC7	test_process_transactions_single_deposit_loop_runs_once	04	L1=1: one iteration, D4 true	Balance=149.95, tx count=1	**PASS**
TC8	test_process_transactions_deposit_then_withdrawal_loop_runs_twice	04,01	L1=2+: two iterations, D4 then D1	Balance=129.90, tx count=2	**PASS**
TC9	test_process_transactions_withdrawal_takes_d1_true_branch	01	D1 true: withdrawal dispatched	Balance=149.95, tx count=1	**PASS**
TC10	test_process_transactions_transfer_takes_d2_true_branch	02	D1 false, D2 true: transfer dispatched	src=399.95, dst=299.95	**PASS**

TC#	Test Function	Code	Decision / Loop	Expected Result	Pass/ Fail
TC11	test_process_transactions_paybill_takes_d3_true_branch	03	D1-D2 false, D3 true: paybill dispatched	Balance=149.95, tx count=1	**PASS**
TC12	test_process_transactions_create_takes_d5_true_branch	05	D1-D4 false, D5 true: create dispatched	Account count=2, new acct exists	**PASS**
TC13	test_process_transactions_delete_takes_d6_true_branch	06	D1-D5 false, D6 true: delete dispatched	Account count=0	**PASS**
TC14	test_process_transactions_disable_takes_d7_true_branch	07	D1-D6 false, D7 true: disable dispatched	Status=D, balance/ count unchanged	**PASS**
TC15	test_process_transactions_change_plan_takes_d8_true_branch	08	D1-D7 false, D8 true: plan change dispatched	Plan=NP, balance/ count unchanged	**PASS**
TC16	test_process_transactions_unknown_code_takes_else_branch	99	D1-D8 false, else true: unknown code	All state unchanged	**PASS**

4. Test Execution Results

All 16 tests were run using pytest from the Phase 4 project root:

```
python -m pytest tests/test_backend.py -v
```



Result: 16 passed in 0.07s

All 16 tests passed on the first run. No unexpected errors or assertion failures occurred.

5. Failures Found

No failures were found. All 16 tests passed. Both methods handled normal inputs, boundary cases (e.g. balance just below the fee threshold, empty transaction list), and invalid inputs (unknown account number, disabled account, unrecognized transaction code) correctly.

Test Name	Expected	Actual	Failure Description
No failures detected.	-	-	-

6. Summary

For _apply_deposit(), 5 test cases covered all identified statements. This included both plan fee branches (SP and NP), both outcomes of the guard condition (missing account, disabled account), and the edge case where the balance is too low for the fee to be deducted.

For process_transactions(), 11 test cases covered the loop at 0, 1, and 2+ iterations, and exercised the true branch of each of the 9 dispatch decisions (D1-D9). False branches are covered implicitly through the elif structure. All 16 tests passed with no defects found in either method.

7. GitHub Repository:

https://github.com/Jollysoni/SQA-2026